

Introduction

Big Two is a popular card game of Cantonese origin, involving strategic play with the objective of being the first to empty one's hand. I will briefly go through the rules of the game for a better understanding of this project. Big Two is a card game that is played by four players using a standard deck of 52 cards. Each player will start with a hand of 13 cards and take turns playing card combinations. Possible combinations are singles, pairs, triples and five card combinations. For five card combinations, there are straights, flushes, full houses, four of a kinds and straight flushes, in ascending order of its size. If there are no cards in play, or everyone passed their turns, the player can choose to play any combinations. If there are cards in play, the player must either play a higher combination with the same number of cards (singles with singles, pairs with pairs etc.) or choose to pass. Game ends when one player empties their hand.

This project aims to create an AI to play big two using reinforcement learning. There are two main objectives to this project:

1. Creating a functional AI to complete a game of big two based on simple rules (Simple AI)
2. Enhance the AI through reinforcement learning methods to allow the AI to play with better strategies and increase its win rate. (Q-Learning AI)

Motivation of this project is to develop an AI that can play a culturally significant game from my place of origin. As big two is not widespread in western countries, this area is less explored. However, there are similar AIs being implemented to play other card games, such as poker AIs, UNO AIs etc.

Methods

This project is implemented in python, both the game and the AI, and an interface also in python is used to connect the AI to the game, and loops through multiple games automatically to allow the AI to learn.

- Objective 1: Simple AI
 - A simple rule-based AI was implemented.
 - This AI receives its hand of cards, and the last card combination played in the game.
 - It scans for all possible plays based on its hand and the last played combination, and either:
 - Chooses card combinations with highest number of cards to play if no one has played or everyone has passed
 - Chooses the largest 5 card combinations to play if it must play 5 cards combinations
 - Chooses the smallest combinations to play if it has to play anything other than 5 cards combinations
 - Chooses to pass if it is unable to play any combinations
- Objective 2: Reinforcement Learning AI
 - A Q-Learning AI was implemented using temporal difference learning with feature-based state representation. Each Q-value is represented by a linear combination of weights and features calculated by using own hand, last played combinations, opponent's hand sizes and the combination the AI choose to play (action)
 - Feature-based Q-Learning is selected as the state space using player hand is too large and unpractical. Using features allows the AI to know what to do in a never-before-seen hand based on past experiences.

- Features used:
 1. Number of cards in hand
 2. Number of cards in the action
 3. Difference of own hand size to the average hand size of all opponents
 4. The validity of the action
 5. Rank of the smallest card in hand
 6. Rank of the largest card in hand
 7. Length of the smallest opponent's and
 8. Length of the largest opponent's hand
 9. Number of pairs left after action
 10. Number of triplets left after action
 11. Number of straights left after action
 12. Number of flushes left after action
 13. Number of full houses left after action
 14. Number of four of a kind left after action
 15. Number of straight flushes left after action
 16. Number of big cards in hand after action
 17. Ratio of small to big cards in hand before action
 18. Ratio of small to big cards in hand after action
 19. Inverse of size of action chosen compared to all valid actions
- Q-value calculation
 - Linear combination of the features with their corresponding weights
 - Discount factor for Q-value used is 0.95
- Rewards
 - Rewards are calculated by the combination of the following values
 - Large + reward for winning the game
 - Large – reward for losing the game
 - Small - reward for passing
 - Exponential + rewards for playing more cards
 - Decent + reward for opponent passing their turn
 - Small – rewards for size of plays for each opponent
 - Decent – reward when opponent is closer to winning
 - Decent – reward for breaking big combinations in own hand after the action is played
- Learning
 - Epsilon-greedy policy is used for selecting action, with an exploration rate of 0.5 and an exploration decay of 0.995.
 - Exploration rate is reduced by 0.005 after each play
 - Weights are updated individually using td error and a gradient of corresponding feature
 - Td error is calculated using

$$\text{td_error} = \text{reward} + \text{discount_factor} * \text{max_future_q_value} + \text{current_q_value}$$
 - Gradient is calculated using tanh on the corresponding feature
 - All weights were initialized to be 1 and were written onto a CSV file after each play.
 - Learning was performed by making the Q-Learning AI play against 3 simple AIs in the custom made big two game.
 - 2 different types of card deals were used:

- Random card shuffle and deal, which is identical to how big two is normally played
 - A predefined card deal, where the hand of each AI is predetermined, and no random shuffling occurs.
- Q-Learning AI is trained for roughly 10000 games using a random deal, and roughly 1000 games for the predefined deal.
- Validation methods
 - Q-Learning AI plays 100 games against 3 simple AIs using a predefined card deal, where the Q-Learning AI gets a large hand that gives the AI advantage. Number of wins was recorded
 - Q-Learning AI plays 100 games against 3 simple AIs using a random card deal for every game. Number of wins was recorded
 - 3 Q-Learning AIs play a couple games against the player. The feedback of the player on the difficulty was recorded
 - For control, a Simple AI is set to perform the same validation against other Simple AIs, and the win rates were also recorded.

Dataset

This project does not have a dataset as it is designed to learn as it plays against other AIs. The game environment is custom-made, and an interface is used to link the game environment with the AI.

Results

A total of 2 main learning trials were performed.

1. Using all the features
2. Using only the first 8 features

Smaller learning trails of using different combinations of features were also performed but yielded similar results to first trail.

In addition, a validation trail using Simple AI was also performed for control purposes.

- Quantitative results
 - Trail 1
 - Playing against 3 simple AIs using a predefined card deal
 - Q-Learning AI was able to win 0 games out of 100 games
 - Playing against 3 simple AIs using random card deals
 - Q-Learning AI was able to win 5 games out of 100 games
 - Trail 2
 - Playing against 3 simple AIs using a predefined card deal
 - Q-Learning AI was able to win 100 games out of 100 games as expected
 - Playing against 3 simple AIs using random card deals
 - Q-Learning AI was able to win 22 games out of 100 games
 - Control Trail
 - Playing against 3 simple AIs using a predefined card deal
 - Q-Learning AI was able to win 100 games out of 100 games
 - Playing against 3 simple AIs using random card deals
 - Q-Learning AI was able to win 24 games out of 100 games
- Qualitative results

- Trail 1
 - Player finds AI easy to defeat.
 - Strategies used by AI do not make sense in general.
- Trail 2
 - Player finds AI moderately challenging to defeat at the start.
 - AI uses non-trivial strategies to play against player, player was unable to find a pattern from the AI.
 - Player does not find the AI to be easier to defeat after a couple games.
- Control Trail
 - Player finds AI moderately challenging to defeat at the start.
 - AI uses trivial strategies to play against player, player was able to find the play pattern from the AI after a short period of time.
 - Player finds the AI to be easier to defeat after a couple games.

Challenges

1. Feature Design
 - Balancing the simplicity, robustness and the effectiveness of the features proved to be challenging.
 - Despite having more features covering more areas of strategies often performed by human players, it led to worst performance for the AI.
 - The added amount of features required a hefty amount of computing power for the AI to run, making learning slower compared to using less features.
2. Learning Stability
 - Weight updates occasionally led to divergence, which resulted in some weights to be calculated as NaN, in turn crashed the code during learning.
 - Gradient clipping and normalization were used when updating the weights to resolve this issue.
3. Effectiveness
 - Despite having a high amount of learning trails across different sets of features and weights, the Q-Learning AI is still not able to get a better win rate compared to a Simple AI.

Discussion

- Limitations
 - Inconsistent Win Rates
 - Despite the learning process, the Q-Learning AI is still not able to obtain a higher win rate compared to the Simple AI
 - The current model only uses a set of limited features to learn to play big two, which is not how human players play the game
 - Feature Dependence
 - The Q-Learning AI heavily relies on the designed feature set. Missing critical features could reduce the effectiveness of the AI drastically.
 - Certain features are computationally intensive, which makes it difficult to scale up the feature set.
 - Unable to be used to learn other card games
 - The Q-Learning AI did not learn to find the valid actions, but instead was fed the valid actions through a specific function from the interface.
 - The AI lacks the ability to learn valid plays from a game, therefore is unable to be used to learn other card games.
- Directions for future work

- Use Neuro Networks to learn the weights
 - Currently the weights were learnt using the gradient, which is not as effective compared to neuro networks
 - Using neuro networks will allow the AI to learn more complex strategies of playing the game
- Add memory and opponent modeling
 - Currently the AI chooses an action based on the state it is currently in
 - Having memory of past played cards and previous games will allow the AI to better predict the opponent's hands, and can generate a model for each opponent, which allows the AI to use more advanced strategies
- Implications of the work
 - This project shows that Q-Learning could be viable for creating AIs that play moderately complex card games.
 - Similar AIs that play other card games can be implemented in a similar fashion.
 - This project deepens my understanding on reinforcement learning, and can serve as a case study for using machine learning in game AI

Conclusion

This project successfully developed an AI using Q-Learning that is capable of playing Big Two. Although the AI did not show any notable improvements after intensive learning, it still shows a decent challenge when playing against human players.

References

1. Big Two (Wikipedia): https://en.wikipedia.org/wiki/Big_two